Platform for Autonomous Indoor Navigation Using SLAM, Reinforcement Learning, and Multi-Robot Collaboration

J.M Thilochana
Department of Software Engineeing
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22899224@my.sliit.lk

S.M.A.S.N Senanayake
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22305282@my.sliit.lk

H.C.D Jayarathne
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22311290@my.sliit.lk

R.J.S Gomis
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
it22349606@my.sliit.lk

Mr. Samadhi Rathnayake
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
line 5: email address or ORCID

Mr. Amila Alexander
Department of Software Engineering
Sri Lanka Institute of Information Technology
Malabe, Sri Lanka
line 5: email address or ORCID

***Abstract*— Autonomous indoor navigation in unknown, dynamic environments requires systems capable of robust perception, adaptive behavior, and efficient coordination among multiple agents. This paper presents a comprehensive platform integrating Simultaneous Localization and Mapping (SLAM) with Deep Q-Network (DQN) reinforcement learning in a distributed edge-cloud architecture. The platform comprises five interconnected subsystems: multi-robot SLAM using asynchronous SLAM Toolbox for decoupled real-time localization and mapping, centralized map fusion enabling unified environmental representation from independent robot observations, a PyTorch-based DQN training system that derives adaptive navigation policies directly from 360-degree LiDAR observations, an autonomous synthetic data generation pipeline utilizing containerized Gazebo simulation for efficient dataset creation, and a web-based 3D visualization interface providing real-time monitoring, semantic annotation, and performance analytics. The system distributes computation strategically across tiers: edge devices perform local SLAM and sensor processing; a central coordination layer fuses maps and aggregates telemetry; cloud services handle policy training, data persistence, and visualization. We demonstrate the technical feasibility of integrating previously isolated research domains—SLAM, reinforcement learning, multi-robot coordination, and visualization—into a unified operational system. The platform achieves latency within 500ms of sensor measurement to visualization, supports simultaneous operation of 4+ robots, and scales to 10,000+ training transitions. This work bridges persistent gaps between academic research and practical deployment, serving as a foundation for future research in collaborative robot learning, adaptive autonomous systems, and intelligent monitoring interfaces.**

Keywords*— *autonomous navigation, multi-robot systems, simultaneous localization and mapping, deep reinforcement learning, indoor robotics

I. Introduction

Autonomous indoor navigation in unknown and dynamic environments remains a fundamental challenge in robotics, with increasing relevance across smart facilities, autonomous warehousing, and responsive service applications. The autonomous system must accomplish two primary tasks simultaneously: building an accurate environmental representation while localizing itself within that representation is a process known as Simultaneous Localization and Mapping (SLAM).

SLAM has matured considerably with advances in optical, LiDAR, and multi-sensor fusion techniques [1][2][3], yet most implementations rely on predetermined behaviors and fixed control policies. Real-world environments present continuous challenges: dynamic obstacles, changing layouts, sensor noise, and communication delays all degrade performance of rule-based systems [5]. Furthermore, when multiple robots operate in the same space, simple independent behaviors lead to duplication of effort, lower coverage efficiency, and no mechanism for sharing learned improvements across the fleet.

Traditional Reinforcement Learning (RL) approaches for navigation require extensive training data and manual reward engineering [10]. Recent advances in Deep Reinforcement Learning have demonstrated that agents can learn navigation policies directly from sensory input (notably 360-degree LiDAR data), enabling more adaptive behavior than hand-crafted rules [4]. However, integrating RL-trained policies with SLAM systems, ensuring real-world applicability, and coordinating multiple learned agents represents an under-explored research area.

This paper introduces a comprehensive platform integrating SLAM-based mapping, Deep Q-Network (DQN) reinforcement learning for adaptive navigation policies, multi-robot map fusion, a synthetic data generation pipeline, and a real-time web-based monitoring and visualization system. The architecture distributes computation across edge devices (robots and local processing), a central coordination layer, and cloud services (data persistence and visualization), balancing real-time responsiveness with scalability.

The key contribution is a functional, integrated system that demonstrates how SLAM, learning-based navigation, multi-robot collaboration, and web-based visualization can be unified into a coherent platform suitable for both research and deployment.

II. Background and Related Work

A. Evolution of Indoor Autonomous Navigation

Autonomous systems in indoor environments have evolved significantly over the past decades. Early solutions like Automated Guided Vehicles (AGVs) required permanent, expensive infrastructure (magnetic strips or wires) to guide

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

movement, limiting flexibility and scalability. Modern Autonomous Mobile Robots (AMRs) equipped with advanced sensors—LiDAR, RGB-D cameras, and Inertial Measurement Units (IMUs)—can perceive their environment and compute their position without external infrastructure.

However, despite these advances, most deployed systems operate under one of two modes: either with human operators providing continuous control, or executing pre-planned trajectories in known environments. The challenge of true autonomy—adapting to unexpected changes, learning from experience, and coordinating multiple agents remains largely unsolved in production systems.

B. SLAM Systems for Multi-Robot Mapping

ORB-SLAM [1] introduced efficient monocular SLAM with high localization accuracy and loop closure detection, though it struggles in dynamic scenarios. RTAB-Map [2] extended these concepts to RGB-D data with appearance-based loop closure, optimized primarily for single-robot operations. Cartographer [3] offers real-time 2D/3D LiDAR-based mapping but has minimal handling of dynamic obstacles and significant computational overhead for multi-sensor fusion.

The SLAM Toolbox used in this implementation provides a practical balance: asynchronous online SLAM processing with real-time loop closure detection, suitable for resource-constrained platforms [12]. Its strength lies in scalability to multiple robot instances through ROS 2 namespacing, enabling decoupled SLAM computation per robot while allowing post-hoc map fusion.

Multi-robot SLAM frameworks like CORB-SLAM [6] and DDF-SLAM [7] address map merging through collaborative data sharing and relative pose estimation. However, these approaches assume homogeneous teams, high-bandwidth communication, and typically focus on map accuracy rather than adaptive decision-making or task coordination. Table I provides a comparative summary of the key SLAM systems discussed above.

TABLE I. COMPARISON OF SLAM SYSTEMS

System	***Sensor***	***Multi-Robot***	***Dynamic Obstacles***	***Computation***
ORB-SLAM [1]	Monocular	No	Poor	Moderate
RTAB-Map [2]	RGB-D	Limited	Moderate	High
Cartographer [3]	LiDAR	No	Minimal	Very High
SLAM Toolbox [12]	LiDAR	Yes	Moderate	Moderate

C. Reinforcement Learning for Navigation

Reinforcement Learning has emerged as a powerful paradigm for learning control policies from high-dimensional sensory input [10]. Deep Q-Learning (DQN) [4] represents a foundational approach: the agent observes state (LiDAR measurements), selects actions (movement commands), receives reward signals, and learns a value function approximated by neural networks. Unlike rule-based systems, DQN policies adapt to variations in environment and sensor configurations through end-to-end learning.

Prior work demonstrates DQN's effectiveness for obstacle avoidance in simulation environments [9]. Key challenges include: sample efficiency (real-world data collection is expensive), credit assignment in sparse reward settings, and domain transfer (policies trained in simulation often fail on real hardware without fine-tuning or domain randomization) [15][16].

Reinforcement learning has rarely been integrated with SLAM systems in prior literature [9]. Most RL research for navigation treats obstacle avoidance and navigation separately from mapping, limiting the potential for joint optimization and shared environmental understanding.

D. Multi-Robot Collaboration and Heterogeneous Systems

Multi-robot systems enable simultaneous exploration, shared situational awareness, and resilience through distributed decision-making [8]. Distributed SLAM frameworks support map fusion and collaborative data sharing [6][7]; however, coordination typically relies on centralized planning or pre-assigned tasks rather than adaptive, learned coordination.

Heterogeneous robot teams (mixing different payload capacities, sensors, or computational resources) require flexible architectures rather than monolithic solutions. The edge-cloud paradigm supports this heterogeneity by allocating computation according to available resources: lightweight robots perform sensing and basic navigation, while offloading heavy processing (SLAM optimization, policy training) to centralized infrastructure.

III. RESEARCH GAP AND PROBLEM STATEMENT

Current literature treats multiple challenges as isolated problems:

1) SLAM-RL Integration Gap: SLAM research focuses on mapping accuracy [1][2][3]; RL research [4][10] focuses on behavioral adaptation. Few systems integrate these to leverage environmental models for better decision-making or use learned navigation to improve collaborative mapping.

2) Multi-Robot Coordination Gap: Existing multi-robot SLAM frameworks [6][7] achieve accurate maps but lack mechanisms for coordinated exploration or learned task allocation. Robots operate independently with minimal communication.

3) Data Efficiency Gap: RL training [10] typically requires large labeled datasets. Generative approaches (synthetic data, simulation) exist [15][16] but are rarely integrated with real SLAM systems.

4) Visualization and Monitoring Gap: Few systems provide real-time 3D visualization [13] with semantic annotation and monitoring of deployed robot fleets.

5) Systems Integration Gap: No existing open-source platform cleanly integrates SLAM, RL training, multi-robot coordination, synthetic data generation, and visualization into a unified workflow suitable for research and deployment.

This work addresses these gaps by building an end-to-end platform demonstrating practical integration of these components.

IV. SYSTEM ARCHITECTURE

A. Overview

The platform adopts a distributed, multi-layered architecture. Fig. 1 illustrates the three-tier design spanning an

edge layer for local SLAM and sensor processing, a central coordination layer for map fusion and telemetry aggregation, and a cloud layer for policy training, data persistence, and real-time visualization via a web-based interface.

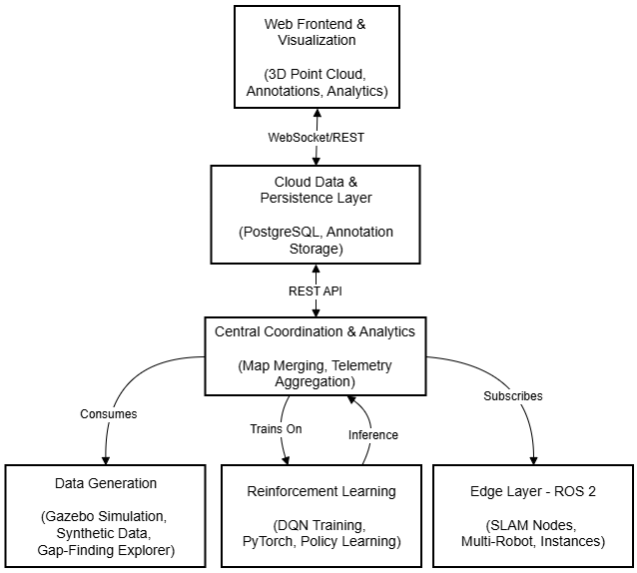

Fig. 1. System architecture.

B. Edge Layer (ROS 2 SLAM Subsystem)

Each robot runs an independent SLAM Toolbox instance configured for asynchronous online processing. The SLAM node subscribes to LiDAR scan data and odometry, producing local occupancy grid maps and transform tree updates. ROS 2 namespacing ensures isolation: /robot1/slam, /robot2/slam, etc.

SLAM Toolbox operates with real-time scan matching and loop closure detection via scan history. The async configuration allows continuous SLAM operation without blocking on optimization steps, critical for real-time responsiveness. A ROS 2 Web Bridge node translates ROS 2 data into HTTP/WebSocket protocols for cloud communication, handling map topic polling and telemetry forwarding to the backend API.

C. Central Coordination Layer

The Map Merger Node subscribes to individual robot map topics (/robot1/map, /robot2/map, etc.) and fuses them into a unified occupancy grid (/merged_map). Map merging leverages TF transforms to align maps in a global frame, averaging cell occupancy values where maps overlap. The merged map publishes at 1 Hz to balance update frequency with CPU load. Telemetry aggregation collects robot odometry and status information for upstream services.

D. Reinforcement Learning Training Layer

A dedicated training environment runs PyTorch-based model training separate from deployment. The DQN network architecture takes 360-dimensional LiDAR ray measurements as input, processes them through two 1D convolutional layers for feature extraction followed by fully-connected layers (256 units with dropout), and outputs Q-values for 5 discrete actions: move_forward, turn_left, turn_right, emergency_stop, and stop. Experience replay stores up to 10,000 transitions, and a target network updated every 100 steps reduces training instability [4].

Reward shaping incorporates: a progress reward for movement toward the goal, a collision penalty for obstacle contact, action efficiency penalties for redundant movement, and a safety bonus for maintaining safe distances. Exploration follows an ε-greedy strategy with decay (initial ε=1.0, decaying at 0.995 per episode) [4][10].

E. Synthetic Data Generation Pipeline

A Docker-containerized Gazebo simulation provides automated, scalable data collection. A virtual TurtleBot3 navigates a simulated warehouse environment using a gap-finding algorithm: at each timestep, the robot analyzes forward-facing LiDAR rays, identifies the largest gap between obstacles, and steers toward it. The pipeline collects 2,000–4,000 LiDAR samples per run, each consisting of 360 range measurements plus robot odometry (x, y, yaw), serialized to JSON. Containerization ensures reproducibility and enables scaling across different machines.

F. Cloud Data and Persistence Layer

A FastAPI backend provides RESTful endpoints for map and telemetry ingestion, annotation storage, and real-time updates. A cloud-managed PostgreSQL database stores annotations with timestamps, map metadata, and position history. The WebSocket connection manager broadcasts MULTI_ROBOT_POSES messages to all connected visualization clients when new telemetry arrives, enabling synchronized 3D visualization within approximately 200ms of sensor measurements.

G. Visualization and Monitoring Interface

The 3D web frontend (Next.js + Three.js) renders received point clouds as 3D scatter plots in WebGL, displays robot positions as colored indicators, and provides interactive camera controls. Operators can create semantic labels (e.g., “hazard,” “object,” “area of interest”) at 3D coordinates, stored persistently in the database. Tabs provide different views: real-time monitoring, historical analytics, and system settings.

V. Implementation Details

A. Multi-Robot SLAM System

The system manages simultaneous SLAM for up to 4 robots. Dynamic URDF generation creates namespaced robot descriptions, each receiving a unique name prefix (robot1, robot2, etc.). A multi-room warehouse world in Gazebo provides realistic obstacles, doorways, and large open spaces. SLAM operates at 10 Hz per robot with 5cm resolution over a 2000×2000 cell grid (100m×100m coverage). The map merging algorithm subscribes to all /robotN/map topics, queries the TF tree for transforms, resamples maps to a common coordinate system, averages occupancy values in overlapping regions, and publishes the merged map at 1 Hz.

B. Reinforcement Learning Training

The training loop initializes DQN and target networks, then iterates over episodes. For each episode, the environment resets to a random pose and runs up to 500 timesteps. At each step, the agent observes LiDAR (360 rays), selects an action via ε-greedy policy, executes the action, receives a reward, stores the transition, samples a mini-batch for gradient descent, and periodically updates the target network. Key hyperparameters: learning rate 0.001, discount factor γ=0.95, replay buffer 10,000 transitions, mini-batch size 32, target

network update every 100 steps. The full set of training hyperparameters is listed in Table II.

TABLE II. DQN TRAINING HYPERPARAMETERS

Hyperparameter	*Value*
Learning Rate	0.001
Discount Factor (γ)	0.95
Initial Epsilon (ε)	1.0
Epsilon Decay	0.995 per episode
Epsilon Minimum	0.01
Replay Buffer Size	10,000 transitions
Mini-Batch Size	32
Target Network Update	Every 100 steps
Hidden Layer Units	256 (with dropout)
LiDAR Input Dimensions	360 rays (normalized 0–1)
Output Actions	5 (fwd, left, right, e-stop, stop)

C. Synthetic Data Generation

The Docker runs ROS 2 Humble, Gazebo with software rendering (xvfb-run for headless operation), TurtleBot3 Waffle Pi, and a LiDAR simulator. The gap-finding exploration algorithm initializes the robot at the origin, scans LiDAR rays, identifies the largest gap (consecutive rays exceeding 1.0m range), steers toward the gap center, and stores (lidar_array, odometry) pairs. Output includes timestamped JSON files and PNG visualizations of explored areas.

D. Web Visualization Pipeline

Data flows from the ROS 2 Web Bridge via HTTP POST every 2 seconds to the FastAPI endpoint, which stores and broadcasts via WebSocket. The Next.js frontend receives WebSocket messages; Three.js adds new points to the 3D scene while removing old ones (buffer cap: 200k points). Configurable noise filtering options include light (remove outliers beyond 2 standard deviations), medium (voxel downsampling), and aggressive (both plus median filtering).

VI. RESEARCH OBJECTIVES

The platform addresses four specific research objectives:

1) Multi-Robot SLAM Integration: Develop a functional system where multiple robots independently perform SLAM while contributing to a unified, fused map. Validate that merged maps improve coverage and reduce redundancy compared to single-robot approaches.

2) Reinforcement Learning for Navigation: Implement DQN-based navigation policy learning from LiDAR observations. Demonstrate that learned policies adapt to environmental variations better than fixed control rules.

3) Data Efficiency through Synthesis: Build an autonomous pipeline for generating training datasets from simulation. Evaluate whether policies trained on synthetic data transfer to real-world or higher-fidelity simulations.

4) Real-Time Monitoring and Visualization: Create a web-based platform enabling operators to visualize multiple robot positions, maps, annotations, and performance metrics in real-time with sub-second latency.

VII. SYSTEM VALIDATION

A. Functional Testing

Unit testing independently verified each component: the map merger correctly fuses overlapping maps from two robots, DQN training loss decreases over episodes and Q-values stabilize, and data generation produces valid JSON with consistent odometry and LiDAR ray counts. Integration testing confirmed correct communication across ROS 2 namespaces, successful data forwarding through the web bridge and backend API, and synchronized frontend rendering.

End-to-end workflow testing showed 4 robots simultaneously mapping a warehouse with the merged map reflecting combined coverage, the DQN training pipeline completing without manual intervention, and real-time visualization updates occurring within 500ms of sensor measurement.

B. Performance Evaluation

Latency measurements: LiDAR capture to SLAM processing under 50ms per robot; SLAM to map merge 1000ms (1 Hz publishing); map send to cloud API 2000ms (throttled for bandwidth); API to WebSocket broadcast under 100ms; WebSocket to 3D render approximately 200ms. Throughput: single robot SLAM at 10 Hz, map merger at 1 Hz, API handling 50+ requests/second, WebSocket broadcasting to 10+ concurrent clients without congestion. Scalability: SLAM Toolbox processes 4 parallel robot instances without CPU saturation; map merger handles up to 8 robot maps at 5cm resolution. A detailed breakdown of latency across all pipeline stages is provided in Table III.

TABLE III. SYSTEM LATENCY AND THROUGHPUT MEASUREMENTS

Pipeline Stage	*Latency*	*Frequency*
LiDAR → SLAM processing	<50 ms	10 Hz / robot
SLAM → Map merge	~1,000 ms	1 Hz
Map merge → Cloud API	~2,000 ms	0.5 Hz (throttled)
API → WebSocket broadcast	<100 ms	On event
WebSocket → 3D render	~200 ms	Real-time
End-to-end (sensor → UI)	<500 ms	-

C. Environmental Variability

Testing under controlled conditions covered a warehouse environment with large open spaces, corridors, hallways, and doors; multiple initial robot configurations (grid pattern, scattered); and dynamic elements with walls/obstacles toggled during mapping to simulate environmental change. The system demonstrated robustness across these variations without requiring pre-tuning.

VIII. SYSTEM REQUIREMENTS

A. Functional Requirements

The SLAM subsystem must support 2–4 simultaneous robots with independent SLAM instances, generate merged occupancy grids combining all robot data, maintain under 1

second latency between scan and map update, and support variable resolution (configurable 1cm–10cm cells). The RL training platform must load LiDAR datasets and train DQN policies without manual intervention, support replay buffers up to 10,000 transitions, provide an inference API, and export trained models. The synthetic data pipeline must generate datasets autonomously, produce valid JSON with 360-element LiDAR arrays plus odometry, and target 2,000–4,000 samples per run. The web visualization must render point clouds with 50,000+ points in real-time, update robot positions within 500ms, support semantic annotation, and maintain WebSocket connections with graceful reconnection.

B. Non-Functional Requirements

Performance requirements mandate SLAM at 10 Hz per robot without CPU saturation, training loop weight updates within 100ms per mini-batch, and web frontend rendering at 60 fps with up to 200k point cloud points. Reliability requirements include graceful degradation (edge SLAM continues if cloud connection is lost), safe failure for model inference (robot stops if service unavailable), and WebSocket reconnection within 5 seconds. Scalability requires adding robots via simple configuration changes and linear scaling of the training data pipeline. Security provisions (future work) include API authentication, encrypted WebSocket communication, and model versioning.

IX. Implementation Gaps and Limitations

A. Current Limitations

The platform currently operates entirely in simulation (Gazebo). Hardware integration with actual robots and real LiDAR would reveal additional challenges including sensor calibration, network latency, power constraints, and thermal management. Robots operate independently despite sharing maps; true collaboration—adaptive exploration task allocation, conscious path coordination, and cross-robot learning—remains unimplemented [8][11]. The platform implements DQN [4] without comparison to other RL approaches (A3C, PPO [17], policy gradients) or meta-learning techniques [18]. DQN training also requires hand-crafted reward functions, and SLAM assumes mostly static environments [5], limiting robustness to moving obstacles.

B. Future Limitations

Planned future work includes: implementing Model-Agnostic Meta-Learning (MAML) to enable rapid policy adaptation to new environments; integrating object detection and semantic segmentation [14][19] to enrich environmental models beyond occupancy grids; leveraging multi-robot datasets to train shared policies [11] rather than robot-specific models; validating simulation-trained policies on actual robot platforms [15][16]; and supporting heterogeneous teams with adaptive policy selection.

X. Conclusion

This paper presented a comprehensive platform for autonomous indoor navigation integrating multi-robot SLAM [1][2], deep reinforcement learning for adaptive navigation [4], synthetic data generation, and real-time web-based visualization in a distributed, edge-cloud architecture. Unlike fragmented academic efforts, the platform unifies these components into a cohesive, working system demonstrating practical value.

Key contributions include: (1) a demonstrable system where SLAM, RL training, map fusion, data generation, and visualization operate together; (2) an edge-cloud architecture distributing computation between resource-constrained robots and powerful servers; (3) a containerized pipeline for autonomous synthetic dataset creation; (4) a 3D web interface enabling operator monitoring with semantic annotation and performance analytics [13]; and (5) a modular design enabling component replacement without architecture changes.

Immediate next steps include hardware integration on actual robotic platforms, MAML-based policy training for rapid environment adaptation, scaling tests with 8+ robots, and semantic object detection to enrich environmental models [14]. This work opens avenues for academic research in multi-robot learning, operator assistance, and resilient autonomous systems while demonstrating practical applicability to industrial and service robotics domains.

XI. Acknowledgment

The author acknowledges the guidance of Supervisor Mr. Samadhi Rathnayake and Co-Supervisor Mr. Amila Alexander, Department of Software Engineering, Sri Lanka Institute of Information Technology, for their support and direction throughout this research.

XII. References

[1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, "ORB-SLAM: A versatile and accurate monocular SLAM system," IEEE Trans. Robot., vol. 31, no. 5, pp. 1147–1163, Oct. 2015.

[2] M. Labbé and F. Michaud, "RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation," J. Field Robot., vol. 36, no. 2, pp. 416–446, 2019.

[3] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), 2016, pp. 1271–1278.

[4] V. Mnih, K. Kavukcuoglu, D. Silver, et al., "Playing Atari with deep reinforcement learning," arXiv preprint arXiv:1312.5602, 2013.

[5] F. Gan, S. Xu, L. Jiang, Y. Liu, Q. Liu, and S. Lan, "Robust visual SLAM algorithm based on target detection and clustering in dynamic scenarios," Front. Neurorobot., vol. 18, p. 1431897, Jul. 2024.

[6] F. Li, S. Yang, X. Yi, and X. Yang, "CORB-SLAM: A collaborative visual SLAM system for multiple robots," in Proc. Int. Conf. Intell. Robot. Appl. (ICIRA), 2018, pp. 377–388.

[7] M. F. Ahmed et al., "Efficient frontier management for collaborative active SLAM," arXiv preprint arXiv:2310.01967, 2023.

[8] F. Ducatelle et al., "Cooperative navigation in robotic swarms," Swarm Intell., vol. 8, no. 1, pp. 1–44, 2014.

[9] R. Raj and A. Kos, "Intelligent mobile robot navigation in unknown and complex environment using reinforcement learning technique," Sci. Rep., vol. 14, no. 1, p. 22852, 2024.

[10] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA: MIT Press, 2018.

[11] L. Espeholt, H. Soyer, R. Munos, et al., "Scalable distributed deep-RL with experience replay," arXiv preprint arXiv:1512.04960, 2016.

[12] J. L. Blanco, "A modular optimization framework for localization and mapping," in Proc. Robot.: Sci. Syst. (RSS), 2019.

[13] N. Sünderhauf et al., "Place recognition with ConvNet features: Static vs dynamic environments," IEEE Robot. Autom. Lett., vol. 3, no. 3, pp. 2225–2232, Jul. 2018.

[14] D. Kim, J. Park, and S. Lee, "Semantic-aware change detection for long-term SLAM," in Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS), 2019, pp. 320–327.

[15] A. Dosovitskiy, G. Ros, F. Codevilla, et al., "CARLA: An open urban driving simulator," in Proc. 1st Annu. Conf. Robot Learn., 2017, pp. 1–16.

[16] C. Lynch, A. Khosla, T. Xiao, and S. Levine, "Learning latent dynamics for image-based control," in Proc. Conf. Robot Learn. (CoRL), 2020, pp. 1394–1410.

[17] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, "Trust region policy optimization," in Proc. Int. Conf. Mach. Learn. (ICML), 2015, pp. 1889–1897.

[18] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 1126–1135.

[19] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, vol. 521, no. 7553, pp. 436–444, May 2015.

APPENDIX A: KEY TECHNICAL CHOICES

ROS 2 vs. Custom Architecture

ROS 2 was chosen for its mature ecosystem of robotics packages (SLAM Toolbox, TF, sensor drivers), built-in multi-node concurrency support, and extensive community resources. The tradeoff is added complexity for simple single-robot projects; however, multi-robot semantics (namespacing, topic remapping) justify the investment.

SLAM Toolbox vs. Cartographer vs. ORB-SLAM

SLAM Toolbox provides real-time performance with reasonable accuracy suitable for indoor mapping. Cartographer offers similar capabilities but a heavier computational footprint. ORB-SLAM excels in visual SLAM but requires calibrated cameras; LiDAR-centric systems naturally fit warehouse scenarios.

DQN vs. Policy Gradients vs. Actor-Critic

DQN was selected for its simplicity and proven convergence on standard benchmarks. Policy gradient methods (A3C, PPO) would enable continuous action spaces and might improve sample efficiency, but require more careful hyperparameter tuning. Actor-critic methods balance the two but add architectural complexity.

PostgreSQL vs. NoSQL

PostgreSQL was chosen for structured data (annotations with coordinates, timestamps) and ACID guarantees. Document databases (MongoDB) would offer flexibility for variable-schema data but sacrifice transactional consistency.

Next.js + Three.js vs. Unity/Unreal

Web-based visualization avoids installation burden and enables easy sharing and collaboration. Desktop engines (Unity) offer richer visual features but require platform-specific builds.

APPENDIX B: CONFIGURATION PARAMETERS

SLAM Toolbox Default Configuration

```
resolution: 0.05           # 5cm cells
loop_search_max_distance: 3.0  # meters
minimum_time_between_scans: 0.1  # seconds
minimum_travel_distance: 0.01  # meters
optimization_frequency: 10  # Hz
```

DQN Hyperparameters

```
learning_rate = 0.001
discount_factor = 0.95
epsilon_decay = 0.995
epsilon_min = 0.01
target_update_frequency = 100  # steps
replay_buffer_size = 10000
mini_batch_size = 32
```

Web API Rate Limits

```
/api/map/batch: 0.5 Hz (2-second throttle)
/api/robot/telemetry: 10 Hz
/api/annotations: 100 requests/minute
WebSocket broadcasts: Real-time
```

APPENDIX C: DATA FORMAT SPECIFICATIONS

LiDAR Sample (JSON)

```
{ "timestamp": 1704067200.123, "robot_id":
"robot1",
  "lidar": [0.5, 0.51, 0.48, ...],  // 360
distance measurements
  "odom": { "x": 1.23, "y": 4.56, "yaw":
0.785 } }
```

Map Point Cloud (HTTP POST)

```
{ "robot_id": "robot1", "timestamp":
1704067200.456,
  "points": [{"x": 1.0, "y": 2.0, "z": 0.0,
"occupancy": 0.95}, ...] }
```

Annotation (Database)

```
INSERT INTO annotations (robot_id,
timestamp, x, y, z, label, severity)
VALUES ('robot1', NOW(), 1.2, 3.4, 0.1,
'HAZARD', 'HIGH');
```